Business Data Mining Homework Session 2

Question 7.3: Predicting Housing Median Prices.
The file BostonHousing.csv contains information on over 500 census tracts in Boston, where for each tract multiple variables are recorded. The last column (CAT.MEDV) was derived from MEDV, such that it obtains the value 1 if MEDV > 30 and 0 otherwise. Consider the goal of predicting the median value (MEDV) of a tract, given the information in the first 12 columns. Partition the data into training (60%) and validation (40%) sets.

This problem primarily uses the k-NN method, which is an instance-based learning algorithm commonly applied to classification tasks. It predicts categories through a voting mechanism among the nearest neighbors. Although k-NN is most frequently used for classification, it can also perform regression prediction by averaging the values of the nearest neighbors. However, it has several limitations in regression contexts, such as sensitivity to outliers and the curse of dimensionality. For low-dimensional problems, k-NN's regression performance can be acceptable. K-NN algorithm operates in an n-dimensional space defined by all features, where n corresponds to the number of independent variables.

CODE:

```
# load data
data(BostonHousing, package = "mlba")

# Skim the data
skimr::skim(BostonHousing)
```

Skimr() is not required for using knn(). But, using skimr() helps to understand data. For example, skimr() helps understand the data distribution, can reveal issues like missing values, outliers and shows if normalization is needed.

```
# Set seed for reproducibility
set.seed(121)
```

Setting a seed ensures reproducible results, enables fair model comparisons, simplifies debugging, and allows other researchers to replicate findings exactly.
I tried using seed (180), and the resulting plot is different from see (121), and the RMSE corresponding to k is also completely different. Different seed values will produce completely different random number sequences, and the answers will also be slightly different.

```
# Partition the data into training (60%) and validation (40%) sets, without column 14
train.index <- sample(1:nrow(BostonHousing), 0.6 * nrow(BostonHousing))
trainData <- BostonHousing[train.index, -14]
validData <- BostonHousing[-train.index, -14]

# Check the partition
dim(trainData)
dim(validData)
```

Dim(trainData) shows there are 303 rows and 13 columns. Dim(validData) shows that there are 203 rows and 13 columns. These commands verify the data partition by checking that the training set contains 60% of samples and validation set 40%, confirming the target variable (column 14) was properly excluded, and ensuring no data loss occurred during splitting.

(a) Perform a k-NN prediction with all 12 predictors (ignore the CAT.MEDV column), trying values of k from 1 to 5. Make sure to normalize the data and choose function knn() from package class rather than package FNN. To make sure R is using the class package (when both packages are loaded), use class: knn(). What is the best k? What does it mean?

From the question (a), we can know that first 12 columns are independent variables and column 13(MEDV) is dependent variables. Column 13 (MEDV) is the continuous target variable for predicting house prices, while the first 12 columns serve as predictor features. Column 14 (CAT.MEDV) is excluded because this is a regression task predicting numerical values rather than categories.

```
# use z-score method to normalize the first 12 columns
normValues <- preProcess(trainData[, -13], method = c("center", "scale"))

# Normalize the data, exclude column 13
trainDataNorm <- trainData
validDataNorm <- validData
trainDataNorm[, -13] <- predict(normValues, trainData[, -13])
validDataNorm[, -13] <- predict(normValues, validData[, -13])
```

Using training set parameters for normalization prevents data leakage and ensures fair model evaluation.

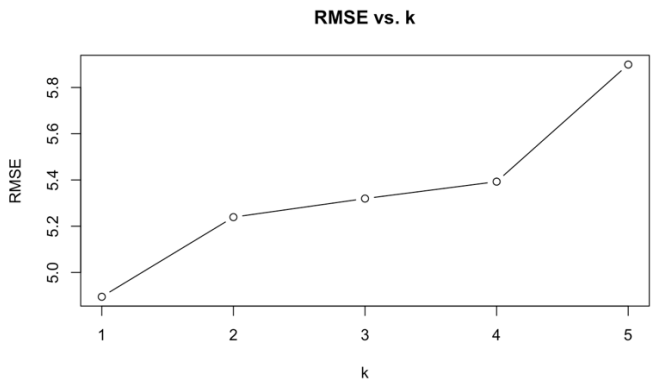
```
# Perform a k-NN prediction with all 12 predictors, k=1,2,3,4,5
accuracy <- data.frame(k = seq(1, 5, 1), RMSE = rep(0, 5))
for (i in 1:5) {
  knn.pred <- class::knn(train = trainDataNorm[, -13],
                         test = validDataNorm[, -13],
                         cl = trainData$MEDV,
                          k = i)
accuracy[i, 2] <- RMSE(as.numeric(as.character(knn.pred)), validData$MEDV)
}

# Find the best k and print the best k
bestK <- accuracy[which.min(accuracy$RMSE), 1]
print(paste("The best k is", bestK))
```

"The best k is 1"

```
# show accuracy table
accuracy
```

k	RMSE
1	4.894865
2	5.184830
3	5.382310
4	5.771486
5	6.151503

```
# Plot the accuracy
plot(accuracy$k, accuracy$RMSE, type = "b", xlab = "k", ylab = "RMSE", main = "RMSE vs. k")
```



According to the accuracy table, k=1 is the best model because it has the lowest prediction error (RMSE = 4.89) on the validation set. RMSE increases and the performance worsens when k increases from 1 to 5.

The best k is chosen by comparing the Root Mean Square Error (RMSE) for different values of k on the validation set. The model with the lowest RMSE is considered the best because it indicates the smallest average difference between the predicted and actual median home prices.

And why k=1 performed best. Follows are the reasons:

The data likely contains many very similar housing tracts. When k=1, the model finds the perfect match and makes an accurate prediction. Using a larger k (like 3 or 5) forces the model to also include less similar neighbors in its calculation. Averaging the prices of these less similar homes introduces error and makes the prediction worse, which is why the RMSE increases.

However, k=1 model is the most complex k-NN model and is highly susceptible to overfitting. It perfectly memorizes the training data, including any noise or outliers. It means the model might be brittle. If we deploy it on new data that doesn't have perfect close matches, its performance could drop significantly.

In conclusion, k=1 is the best because the data has strong local patterns where the price of the single most similar house is a better predictor than the average price of several somewhat similar houses. However, the best practice would be to test this with different training/validation splits to see if k=1 remains the best, as its top performance can sometimes be a result of a particular lucky data split. In practice, we should be cautious about using k=1 as the final model due to its high risk of overfitting and sensitivity to noise, even if it shows the best performance on a single validation set.

(b) Predict the MEDV for a tract with the following information, using the best k:

CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	LSTAT
0.2	0	7	7	0.538	6	62	4.7	4	307	21	10

CODE：

```
# create a new data frame of the new data
newData <- data.frame(CRIM = 0.2, ZN = 0, INDUS = 7, CHAS = 0, NOX = 0.538, RM = 6, AGE = 62, DIS = 4.7, RAD = 4, TAX = 307,
PTRATIO = 21, LSTAT = 10)
bestK <- 1

# normalise the new data using the same scaler as the training data
newDataNorm <- predict(normValues, newData)

# predict the MEDV for the new data using the best k
knn.pred <- class::knn(train = trainDataNorm[, -13],
                       test = newDataNorm,
                       cl = trainData$MEDV,
                        k = bestK)
print(paste("The predicted MEDV is", knn.pred))
```

"The predicted MEDV is 20.4"

The predicted value of 20.4 is the estimated median value of owner-occupied homes.

(c) If we used the above k-NN algorithm to score the training data, what would be the error of the training set?

CODE:

```
# Predict the MEDV for the training data
knn.pred <- class::knn(train = trainDataNorm[, -13],
                       test = trainDataNorm[, -13],
                       cl = trainData$MEDV,
                        k = 1)
print(paste("The RMSE of the training set is", RMSE(as.numeric(as.character(knn.pred)), trainData$MEDV)))
```

"The RMSE of the training set is 0"

Although technically feasible, scoring the training set with the optimal k value is practically meaningless because it inevitably leads to overfitted results.
Predicted_MEDV[i] = Actual_MEDV[i] makes every prediction error (Predicted - Actual) = 0
consequently, RMSE = sqrt(mean(0^2)) = 0
When k=1 in k-NN, the training RMSE is always 0 because each point perfectly predicts itself as its own nearest neighbor. This demonstrates perfect memorization of the training data but does not reflect real predictive ability. We cannot determine whether the model has learned genuine patterns or simply memorized training samples. And good training set performance may falsely suggest the model is ready for real-world deployment.

(d) Why is the validation data error overly optimistic compared to the error rate when applying this k-NN predictor to new data?

By using the validation set to select the optimal parameter (k=1), we introduced a form of data leakage—where information from the validation set influenced model design. This process increases the risk of overfitting, as the chosen model may capitalize on random patterns specific to this 60/40 data split, patterns that may not generalize to new samples from the same population or future data.
Moreover, the k-NN method used here does not account for interactions or correlations among the independent variables. It treats each predictor as separate and equally important, ignoring potential underlying relationships between them. At the same time, the model relies heavily on the local similarity between observations. While this can be useful, it also means the model is highly sensitive to the specific training-validation structure and may fail when that structure changes.

(e) If the purpose is to predict MEDV for several thousands of new tracts, what would be the disadvantage of using k-NN prediction? List the operations that the algorithm goes through in order to produce each prediction.

Here are the disadvantages of larger-scale prediction. All computation is deferred until prediction time. To predict one new tract, the algorithm must compute its distance to every single tract in the training set. For thousands of new predictions, this becomes computationally prohibitive. The entire training dataset (all features and target values) must be loaded into memory and stored permanently to make any future prediction. This is inefficient for large training sets. The time to produce a single prediction is proportional to the size of the training data. Making thousands of predictions would be very slow compared to other models (like linear regression) that generate a prediction almost instantly after being trained. The algorithm uses all predictors. If some features are irrelevant for predicting MEDV, they still contribute to the distance calculation, potentially reducing prediction accuracy without feature selection.

Here are the steps.

step 1 Data Preprocessing

- Standardize the training set
- Apply the same normalization parameters to the test set and cannot standardize training and test sets together. It can prevent data leakage.

step 2 Parameter Selection

- Determine the value of k
- Define the distance metric to be used

step 3 Distance Calculation

- Calculate distances to all training points using chosen metric
- We cannot use only Euclidean distance to calculate the distance. If there are obstacles between two points, we should use Manhattan distance to calculate. For example, food delivery through city blocks where buildings block direct paths
- Common distance formulas:
    1. Euclidean Distance: $d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$
    2. Manhattan Distance: $d(x, y) = \sum_{i=1}^{n} | x_i - y_i |$

step 4 Neighbor Identification

- Sort all training instances by distance to the test point
- Select the k closest neighbors

step 5 Weight Assignment (optional, but I think it is necessary in practical applications)

- Assign weights to neighbors, typically giving higher weight to closer points
- $weight = 1/distance$

step 6 Feature Weighting (optional, but I think it is necessary in practical applications)

- Apply different weights to different features based on their importance
- Requires domain knowledge or feature importance analysis

step 7 Prediction Generation

- For classification: Majority vote among k neighbors, and I think should consider weight.
- For regression: Average or weighted average of neighbors' target values

Question 5.6: A firm that sells software services has been piloting a new product and has records of 500 customers who have either bought the services or decided not to. The target value is the estimated profit from each sale (excluding sales costs). The global mean is $2128. However, the cost of the sales effort is not cheap—the company figures it comes to $2500 for each of the 500 customers (whether they buy or not). The firm developed a predictive model in hopes of being able to identify the top spenders in the future. The lift and decile charts for the validation set are shown in Figure 5.13.

(a) If the company begins working with a new set of 1000 leads to sell the same services, similar to the 500 in the pilot study, without any use of predictive modeling to target sales efforts, what is the estimated profit?

CODE:

```
# known
global_mean_profit_per_sale <- 2128

# Average profit per sale (excluding sales costs)
cost_per_sales_effort <- 2500

 # Sales cost per customer
num_new_customers <- 1000

# Average net profit per customer
net_per_customer <- global_mean_profit_per_sale - cost_per_sales_effort

# Total profit estimate
total_profit <- num_new_customers * net_per_customer

# Output
cat("Average net profit per customer: $", net_per_customer, "\n")
cat("Estimated total profit for 1000 customers: $", total_profit, "\n")
```

```
Average net profit per customer: $ -372
Estimated total profit for 1000 customers: $ -372,000
```

Global average profit per customer (from actual sales) = $2,128

Sales cost per customer (whether they buy or not) = $2,500

Net result per customer: $2,128 - $2,500 = -$372

The company now has 1,000 new leads similar to the pilot group.

Expected net profit per customer = $2,128 - $2,500 = -$372

Total estimated profit = 1,000 × (-$372) = -$372,000

This demonstrates why predictive modeling is valuable, targeting only high-potential customers, the company can avoid the $2,500 sales cost on unlikely buyers and potentially turn a profit.

(b) If the firm wants the average profit on each sale to at least double the sales effort cost, and applies an appropriate cutoff with this predictive model to a new set of 1000 leads, how far down the new list of 1000 should it proceed (how many deciles)?

I think this model is probably valid, but it's not perfect. Column 6 is higher than column 5, and column 8 is higher than column 7. When the overall sample size is small or the number of responding customers in each decile is small, one or two additional responding customers can significantly increase the average response of that decile.

The global mean profit from the pilot data = $2,128.

The target mean profit = $2,500* 2 = $5,000.

The required lift is = $5,000 / $2,128 ≈ $\boxed{2.35}$

This means we need to find the group of leads whose average profit is 2.35 times the global average.

The provided decile-wise lift chart shows the "Mean Response" for each percentile.

The chart indicates that the highest mean response value is around 2.0.
This corresponds to an average profit: $2,128 × 2.0 = $4,256

The highest achievable mean response (2.0) is less than the required mean response (2.35), it is impossible to reach the $5,000 average profit goal using this predictive model. Even if the company contacts only the top 10% of leads, the expected average profit is only $4,256. Adding any more leads from lower deciles will only reduce this average profit further.

In my opinion, the company should not proceed with any deciles based on this model if it must strictly meet its profit goal. The predictive model is not powerful enough to identify a segment of leads with a high enough average profit to meet the "double the sales cost" target. The firm could develop a better predictive model, find ways to significantly reduce the $2,500 sales cost or lower its profit expectation per sale.

(c) Still considering the new list of 1000 leads, if the company applies this predictive model with a lower cutoff of $2500, how far should it proceed down the ranked leads, in terms of deciles?

The company should proceed down to the 4th decile (the top 40% of leads).
Target Mean Response = $2,500 / $2,128 ≈ 1.175
The average profit for the cumulative group up to the 4th decile is above 1.175. In my opinion, I think the 5th and 6th decile is lower than 1.175. Including the 5th decile would cause the cumulative average to drop below the 1.175. Therefore, contacting the top 40% of leads ensures that the average profit per sale meets or exceeds the $2,500 cutoff, while maximizing the number of customers targeted.
The company should apply the model to rank the 1,000 new leads and then only contact the top 400 leads (40%) for its sales campaign.

Question 10.1: Financial Condition of Banks

(a) Write the estimated equation that associates the financial condition of a bank with its two predictors in three formats

In my opinion, I believe we should perform standardization first and then process the data. And we should expand the sample size; 20 samples are too small. Although the independent variables are ratio. I think different ratios have different ranges and degrees of variation. Standardization is useful.
The initial model produced extremely large coefficients and unrealistic odds ratios (e.g., 4321 for Loans/Assets). This typically indicates the need for predictor standardization, especially when variables have different scales - even though both are asset ratios. Standardization would be the correct approach to resolve this and make coefficients comparable. I proceeded with the unnormalized model for this analysis to directly demonstrate this common issue in logistic regression and then use normalized model.

CODE:

```
# train logistic regression model with 2 predictors: TotLns&Lses/Assets and TotExp/Assets, target: Financial Condition
model <- glm(Financial.Condition ~ TotLns.Lses.Assets + TotExp.Assets, data = Banks, family = "binomial") # family = "binomial" means logistic regression, other options are "gaussian" for linear regression, "poisson" for poisson regression, etc.

# check model
summary(model)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(>|z|)
(Intercept)	-14.7210	6.675	-2.205	0.0274
TotLns.Lses.Assets	8.3713	5.779	1.449	0.1474
TotExp.Assets	89.8339	47.781	1.880	0.0601

(Dispersion parameter for binomial family taken to 1)
Null deviance: 27.726 on 19 degrees of freedom
Residual deviance: 13.148 on 17 degrees of freedom

The null deviance of 27.726 represents the deviance of the intercept, with 19 degrees of freedom, indicating the baseline model's fit. The residual deviance of 13.148 represents the deviance of the full model containing all predictors, with 17 degrees of freedom (20 observations - 1 intercept - 2 predictors = 17), reflecting the complete model's fit. The deviance reduction of 14.6 indicates that adding predictors significantly improves model fit, and since the residual deviance (13.148) is less than the degrees of freedom (17), there is no evidence of overdispersion in the model.

TotLns.Lses.Assets:
The positive coefficient (8.3713) suggests that a higher loans-to-assets ratio is associated with an increased risk of financial weakness. However, with a high standard error (5.779) and a p-value of 0.1474, we lack sufficient statistical evidence to confirm this relationship is real and not due to random chance.

TotExp.Assets:
This variable shows a very large positive coefficient (89.8339), indicating a potentially strong association with financial weakness. However, its standard error is exceptionally high (47.781), resulting in a p-value (0.0601) that falls just above the significance threshold. This means the estimate is highly unstable, and we cannot be confident in this apparent effect.

Intercept:
The significant intercept (p = 0.0274) indicates that the baseline log-odds of being financially weak is significantly negative when both predictors are zero, which is logically consistent.

I think the reasons for this situation are as follows:

With only 20 banks, the model has small sample size. The two financial ratios are likely highly correlated. When predictors are correlated, it becomes difficult for the model to isolate their individual effects, leading to inflated standard errors for both. The predictors are on different scales. TotExp.Assets likely has a very small range, forcing its coefficient to be artificially large to have any impact on the log-odds, which in turn increases its standard error.

```
# Extract coefficients
coefficients <- coef(model)

# Intercept
beta0 <- coefficients[1]

# Coefficient for TotLns.Lses.Assets
beta1 <- coefficients[2]

# Coefficient for TotExp.Assets
beta2 <- coefficients[3]

cat("Model Coefficients:\n")
cat("Intercept (β0) =", round(beta0, 4), "\n")
cat("TotLns&Lses/Assets (β1) =", round(beta1, 4), "\n")
cat("TotExp/Assets (β2) =", round(beta2, 4), "\n\n")
```

Model Coefficients:

Intercept (β_0) = - 14.7210

TotLns&Lses/Assets (β_1) = 8.3713

TotExp/Assets (β_2) = 89.8339

However, when I do standard normalization, the situation has some different.

```
#normalize the data
data_normalized <- as.data.frame(scale(Banks[, c("TotLns.Lses.Assets", "TotExp.Assets")]))
data_normalized$Financial.Condition <- Banks$Financial.Condition

#logist regression model
model <- glm(Financial.Condition ~ TotLns.Lses.Assets + TotExp.Assets, data = data_normalized, family = "binomial")

# check model
summary(model)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(>|z|)
(Intercept)	-0.07199	0.74788	-0.096	0.0274
TotLns.Lses.Assets	1.33756	0.92333	1.449	0.1474
TotExp.Assets	2.34031	1.24476	1.880	0.0601

(Dispersion parameter for binomial family taken to 1)
Null deviance: 27.726 on 19 degrees of freedom
Residual deviance: 13.148 on 17 degrees of freedom

After standardization, the results are now more meaningful and interpretable. The statistical significance (z and p-values) for the predictor variables TotLns.Lses.Assets and TotExp.Assets remains unchanged. Their coefficients and standard errors have been transformed to a comparable and sensible scale.
The change in the z-value for the intercept after standardization is normal and expected. It signifies that model has shifted from a difficult-to-interpret baseline to one with a clear business meaning—the risk of an "average bank."

i. The logit as a function of the predictors

logit(p) = log($\frac{p}{1-p}$)= -0.07199 + 1.33756 * TotLns&Lses/Assets + 2.34031 * TotExp/Assets

ii. The odds as a function of the predictors

odds =$e^{-0.07199 + 1.33756 * \text{TotLns\&Lses/Assets} + 2.34031 * \text{TotExp/Assets}}$

iii. The probability as a function of the predictors

p(weak) =$\frac{odds}{1+odds}$ = $\frac{e^{-0.07199 + 1.33756 * \text{TotLns\&Lses/Assets} + 2.34031 * \text{TotExp/Assets}}}{1+e^{-0.07199 + 1.33756 * \text{TotLns\&Lses/Assets} + 2.34031 * \text{TotExp/Assets}}}$

(b) Consider a new bank whose total loans and leases/assets ratio = 0.6 and total expenses/assets ratio = 0.11. From your logistic regression model, estimate the following four quantities for this bank (use R to do all the intermediate calculations; show your final answers to four decimal places): the logit, the odds, the probability of being financially weak, and the classification of the bank (use cutoff = 0.5).

CODE:

```
# 1. New bank data
new_TotLns <- 0.6
new_TotExp <- 0.11

# 2. Calculate logit
logit_value <- beta0 + beta1 * new_TotLns + beta2 * new_TotExp

# 3. Calculate odds
odds_value <- exp(logit_value)

# 4. Calculate probability of being financially weak
probability_value <- exp(logit_value) / (1 + exp(logit_value))

# 5. Classification (cutoff = 0.5)
classification <- ifelse(probability_value >= 0.5, "weak", "strong")

# 6. Display results with 4 decimal places
cat("Estimated quantities for the new bank:\n")
cat("Logit:", round(logit_value, 4), "\n")
cat("Odds:", round(odds_value, 4), "\n")
cat("Probability of being financially weak:", round(probability_value, 4), "\n")
cat("Classification:", classification, "\n")
```

```
Estimated quantities for the new bank:
Logit: 0.988
Odds: 2.6858
Probability of being financially weak: 0.7287
Classification: weak
```

```
# Optional: Display the actual calculations
cat("\nCalculation details:\n")
cat("Logit =", round(beta0, 4), "+", round(beta1, 4), "* 0.6 +", round(beta2, 4), "* 0.11\n")
cat("      =", round(logit_value, 3), "\n")
```

```
Calculation details:
Logit = -0.072 + 1.3376 * 0.6 + 2.3403 * 0.11 = 0.988
```

(c) The cutoff value of 0.5 is used in conjunction with the probability of being financially weak. Compute the threshold that should be used if we want to make a classification based on the odds of being financially weak, and the threshold for the corresponding logit.

CODE:

```
# Given probability cutoff
prob_cutoff <- 0.5

# 1. Calculate corresponding odds threshold
odds_threshold <- prob_cutoff / (1 - prob_cutoff)

# 2. Calculate corresponding logit threshold
logit_threshold <- log(odds_threshold)

# Display results
cat("Threshold values corresponding to probability cutoff = 0.5:\n")
cat("Probability threshold:", round(prob_cutoff, 4), "\n")
cat("Odds threshold:", round(odds_threshold, 4), "\n")
cat("Logit threshold:", round(logit_threshold, 4), "\n")
```

```
Threshold values corresponding to probability cutoff = 0.5:
Probability threshold: 0.5
Odds threshold: 1
Logit threshold: 0
```

```
# Verification
cat("\nVerification:\n")
cat("Probability from odds:", round(odds_threshold / (1 + odds_threshold), 4), "\n")
cat("Odds from logit:", round(exp(logit_threshold), 4), "\n")
```

```
Verification:
Probability from odds: 0.5
Odds from logit: 1
```

Probability cutoff = 0.5
Corresponding odds cutoff = 1
Corresponding logit cutoff = 0
These values remain the same regardless of whether predictors are standardized because they represent the same classification boundary in different measurement scales.

(d) Interpret the estimated coefficient for the total loans & leases to total assets ratio (TotLns&Lses/Assets) in terms of the odds of being financially weak.

CODE:

```
# Extract coefficient and calculate odds ratio
beta1 <- coef(model)[2]
odds_ratio <- exp(beta1)

cat("Interpretation:\n")
cat("The coefficient for Total Loans & Leases/Assets ratio is", round(beta1, 4), ".\n")
cat("The corresponding odds ratio is", round(odds_ratio, 4), ".\n\n")

if (beta1 > 0) {
  cat("This means: For each one-unit increase in the Total Loans & Leases/Assets ratio, ")
  cat("the odds of a bank being financially weak multiply by", round(odds_ratio, 4), ".\n")
  cat("In other words, higher loan ratios are associated with greater odds of financial weakness.")
} else if (beta1 < 0) {
  cat("This means: For each one-unit increase in the Total Loans & Leases/Assets ratio, ")
  cat("the odds of a bank being financially weak multiply by", round(odds_ratio, 4), ".\n")
  cat("In other words, higher loan ratios are associated with lower odds of financial weakness.")
} else {
  cat("This means: The Total Loans & Leases/Assets ratio has no significant effect on the odds of financial weakness.")
}
```

Interpretation:
The coefficient for Total Loans & Leases/Assets ratio is 1.3376.
The corresponding odds ratio is 3.8097.
This means: For each one-unit increase in the Total Loans & Leases/Assets ratio, the odds of a bank being financially weak multiply by 3.8097.
In other words, higher loan ratios are associated with greater odds of financial weakness.

The coefficient of 1.3376 for the standardized loan-to-assets ratio corresponds to an odds ratio of approximately 3.8097. This indicates that a one standard deviation increases in a bank's loans and leases relative to its assets is associated with nearly a fourfold increase in the odds of it being in weak financial condition. This suggests that higher lending exposure, relative to the typical variation observed in the data, is a substantial risk factor for financial weakness.

(e) When a bank that is in poor financial condition is misclassified as financially strong, the misclassification cost is much higher than when a financially strong bank is misclassified as weak. To minimize the expected cost of misclassification, should the cutoff value for classification (which is currently at 0.5) be increased or decreased?

This question describes a scenario with asymmetric misclassification costs:
Case 1**:** A weak bank misclassified as strong → cause large loss
Case 2: A strong bank misclassified as weak → There will be no loss
The cutoff value should be decreased. By lowering the threshold, the model will be more inclined to classify banks as "weak", which will reduce the loss, but due to the cost asymmetry, the overall expected cost will decrease.